



Aula 02: Estrutura de uma Classe Java

Classes em programação orientada a objetos são abstrações de objetos, ou seja, representam de forma genérica as instâncias dos objetos.

A estrutura de uma classe é dividida nome da classe, atributos e métodos.

ContaBancaria
- numero: int - saldo: double - titular: String
+ depositar(double valor, int numero) : boolean + sacar(double valor, int numero) : boolean

Algumas orientações e boas praticas para programação orientada a objetos:

- O nome da classe sempre inicia com maiúscula. Se o nome for formado por mais de uma palavra utiliza-se **UpperCamelCase**.

```
public class ContaBancaria {  
  
    //Atributos  
    private int numero;  
    private double saldo;  
    private String titular;  
  
    // Métodos  
    public boolean depositar(double valor, int numero){  
        if(this.numero != numero)  
    }
```

- **Atributos** e **métodos** sempre iniciam com minúscula. Se o nome for formado por mais de uma palavra utiliza-se **lowerCamelCase**.
- Atributos possuem modificadores de acesso **private** para atender os princípios de encapsulamento, vide próxima sessão.
- Parâmetro de métodos devem iniciar com letras minúsculas. Se o nome do parâmetro for formado por mais de uma palavra, utiliza-se **lowerCamelCase** ou **camelCase**.

```

        return false;
        this.saldo+=valo
r;
        return true;
    }

    public boolean sacar
(double valor, int numer
o){
        if(this.saldo<val
or || this.numero!=numer
o)
            return false;
        this.saldo-=valo
r;
        return true;
    }
}

```

Existem alguns métodos padrões que são característicos do paradigma de programação orientada a objetos. Estes métodos padrões serão apresentados a seguir:

▼ Método Construtor

```

public class ContaBancar
ia {

    //Atributos
    private int numero;
    private double sald
o;
    private String titul
ar;

    //Construtor
    public ContaBancaria

```

Em programação orientada a objetos (POO), um método construtor é um tipo especial de método que é chamado automaticamente quando uma instância de uma classe é criada. O objetivo principal de um método construtor é inicializar os atributos de um objeto, garantindo que ele esteja em um estado válido e consistente desde o momento de sua criação.

```

(int numero, String titular){
    this.numero=numero;
    this.saldo=0;
    this.titular=titular;
}

// Métodos
public boolean depositar(double valor, int numero){
    if(this.numero!=numero)
        return false;
    this.saldo+=valor;
    return true;
}

public boolean sacar(double valor, int numero){
    if(this.saldo<valor || this.numero!=numero)
        return false;
    this.saldo-=valor;
    return true;
}

```

Características importantes dos métodos construtores em POO:

1. **Nome do Método:** O nome do método construtor deve ser o mesmo que o nome da classe onde ele está definido. Em Java, por exemplo, o método construtor é declarado usando a palavra-chave `public` seguida pelo nome da classe.
2. **Inicialização de Atributos:** O método construtor é usado para inicializar os atributos de um objeto. Isso pode envolver a atribuição de valores iniciais aos atributos da classe ou a execução de outras operações necessárias para configurar o objeto.
3. **Execução Automática:** O método construtor é chamado automaticamente quando uma nova

instância da classe é criada com a palavra-chave `new`. Isso garante que o objeto seja inicializado corretamente antes de ser usado em outras partes do programa.

4. Construtor Padrão:

Se nenhum construtor é explicitamente definido em uma classe, o compilador Java fornece um construtor padrão sem argumentos. Esse construtor padrão simplesmente chama o construtor da superclasse e não faz nenhuma inicialização adicional.

▼ Método `toString()`

O método `toString()` é um método especial em muitas linguagens de programação orientadas a objetos, como Java, que é usado para retornar uma representação de string legível para humanos de um objeto. O objetivo principal do método `toString()` é fornecer uma descrição textual do estado do objeto, geralmente para fins de

```
// toString
@Override
public String toString() {
    return "ContaBancaria{" +
        "numero" +
        "= " + numero +
        ", saldo
```

depuração ou exibição de informações para o usuário.

Implementação Personalizada:

Embora a implementação padrão do método `toString()` na classe **Object** retorne uma string que consiste no nome da classe seguido do endereço de memória do objeto, é comum e muitas vezes desejável substituir esse comportamento padrão com uma implementação personalizada que forneça informações mais úteis sobre o estado do objeto como por exemplo os valores de seu atributo.

O método `toString()` é frequentemente usado em situações onde uma representação de string do objeto é necessária, como ao imprimir o objeto no console, ao registrá-lo em logs de depuração ou ao exibi-lo em interfaces de usuário.

```
= " + saldo +  
    ", titul  
ar = " + titular + '\n' +  
    '}'  
}
```

Neste código, como estamos substituindo o método `toString()` da classe `Object` por uma versão personalizada, precisamos fornecer uma anotação de que o método da superclasse será sobrescrito.

A notação `@Override` é uma anotação em Java que indica explicitamente que um método está sendo sobrescrito da superclasse. Isso significa que o método na subclasse tem o mesmo nome, tipo de retorno e lista de argumentos que um método na superclasse.

▼ Encapsulamento

O encapsulamento é um dos princípios fundamentais da programação orientada a objetos, que visa ocultar os detalhes internos de uma classe e expor apenas as interfaces necessárias para interagir com ela. Isso significa que os atributos de uma classe são geralmente declarados como privados, impedindo o acesso direto a eles de fora da classe.

Em Java, usamos os **modificadores de acesso** para controlar a visibilidade dos membros (atributos e métodos) de uma classe. Os principais modificadores são:

1. **public** – O membro pode ser acessado de qualquer lugar.
2. **protected** – O membro pode ser acessado apenas pela própria classe e por classes derivadas.
3. **private** – O membro é acessível apenas dentro da própria classe.

▼ Getters and Setters

O acesso aos atributos de uma classe é fornecido por meio de **métodos públicos**, conhecidos como **getters** e **setters**.

Esses métodos permitem que os usuários da classe leiam e modifiquem os atributos de maneira controlada, garantindo a consistência e integridade dos dados.

```
// Getters and Setters

    public int getNumero() {
        return numero;
    }

    public void setNumero(int numero)
    {
        this.numero
        = numero;
    }

    public double getSaldo() {
        return saldo;
    }
```



Por que utilizar encapsulamento?

O encapsulamento promove o princípio de esconder a implementação interna de uma classe, facilitando a manutenção do código, melhorando a segurança e promovendo uma maior modularidade e reutilização de código.

A ausência de encapsulamento pode levar a uma série de problemas em um programa, incluindo:

1. **Exposição de Dados Sensíveis:** Se os atributos de uma classe não forem encapsulados e forem acessíveis diretamente por outras partes do programa, os dados sensíveis podem ser expostos, comprometendo a segurança do sistema.

```

0;
    }

    public void set
Saldo(double saldo)
{
    this.saldo
= saldo;
}

    public String g
etTitular() {
    return titu
lar;
}

    public void set
Titular(String titu
lar) {
    this.titula
r = titular;
}

```

Exemplo: uma vez criada uma conta bancária, um número é atribuído à ela. A regra de negócio de sistemas bancários deve impedir que este número nunca seja alterado.

Em Java existe uma forma alternativa mais enxuta de se criar os métodos getters e setters. O código abaixo tem o mesmo efeito que o código anterior

2. Fragilidade do Código:

Sem encapsulamento, os detalhes internos de uma classe ficam expostos e podem ser facilmente alterados, o que pode levar a mudanças inesperadas e quebras de outras partes do código que dependem desses detalhes.

3. Acoplamento

Excessivo: Quando os atributos de uma classe são acessíveis diretamente por outras partes do programa, isso cria um forte acoplamento entre as partes do sistema. Isso torna o código mais difícil de entender e manter, pois pequenas alterações em uma classe podem ter um impacto em várias outras partes do sistema.

4. Falta de Reutilização de

Código: Sem encapsulamento, a reutilização de código se torna mais difícil, pois partes específicas de uma classe podem ser difíceis de extrair e

```
public class ContaBancaria{  
    // Atributos  
    public int Numero {get; private set;}  
    public double Saldo {get; set;}  
    public string Titular {get; set;}  
}
```

reutilizar em outras partes do programa.

5. **Dificuldade de**

Depuração: Quando ocorrem erros em um programa sem encapsulamento, pode ser mais difícil rastrear a causa raiz do problema, já que os atributos podem ser modificados de várias partes do código.

6. **Dificuldade de**

Manutenção: Sem encapsulamento, o código tende a se tornar mais difícil de entender e manter ao longo do tempo, pois os detalhes internos de uma classe estão expostos e podem ser modificados livremente por outras partes do programa.

Em resumo, o encapsulamento é fundamental na programação orientada a objetos para promover a segurança, a modularidade, a reutilização de código e a manutenibilidade do software.

▼ Sobrecarga de Métodos

```
public class ContaBancaria {  
  
    // Atributos  
    private int numero;  
    public String titular;  
    private double saldo;  
  
    // Construtor  
    public ContaBancaria(int numero, String titular) {  
        this.numero = numero;  
        this.titular = titular;  
        this.saldo = 0.0;  
    }  
  
    // Sobrecarga de construtor  
    public ContaBancaria(int numero, String titular, double bonus) {  
        this(numero, titular);  
        this.saldo = bonus;  
    }  
  
    // Getters
```

A sobrecarga de métodos consiste basicamente em criar variações de um mesmo método, ou seja, a criação de dois ou mais métodos com nomes totalmente iguais em uma classe.

A Sobrecarga permite que utilizemos o mesmo nome em mais de um método contanto que suas listas de argumentos sejam diferentes para que seja feita a separação dos mesmos.

Construtores também podem ser sobrecarregados.

ContaBancaria
-Numero: int -Saldo: double -Titular: string
+ContaBancaria(numero: int, titular: string) : void +ContaBancaria(numero: int, titular: string, bonus: double) : void +depositar(valor: double) : void +depositar(valor: double, numero: int) : bool +sacar(valor: double, numero: int) : bool

? Por que usar sobrecarga?

- **Legibilidade:** em vez de inventar vários nomes (`SomarInt`, `SomarDouble`, `SomarTresNumeros`), você usa o mesmo nome `Somar`.
- **Flexibilidade:** o mesmo método se adapta a diferentes tipos de entrada.

```

        public int getNumero() {
            return numero;
        }

        public double getSaldo() {
            return saldo;
        }

        // Métodos
        public void depositar(double valor)
        {
            saldo += valor;
        }

        public boolean depositar(double valor, int numero) {
            if (this.numero != numero || valor <= 0) {
                return false;
            }

            saldo += valor;
            return true;
        }

        public boolean sacar(double valor, int numero) {

```

- **Organização:** evita repetição de código desnecessária.

```
        if (this.numero != numero || saldo < valor || valor <= 0) {
            return false;
        }

        saldo -= valor;
        return true;
    }

    // ToString
    @Override
    public String toString() {
        return "Número: " + numero +
            "\nTitular: " + titular +
            "\nSaldo: " + saldo;
    }
}
```

▼ Lista de exercícios 02: POO



O objetivo desta lista de exercícios é treinar a habilidade de converter classes abstratas em código.

attachment:ba5a3093-b76a-423f-97b4-65c30b40bf33:lista2_po.pdf

Instruções:

1. Baixe o arquivo **markdown** abaixo:

lista2_poo.md

2. Abra o arquivo em um editor de arquivos **markdown** com suporte a **mermaid** (sugestão: **VS Code**).
 3. Para cada projeto proposto no arquivo **markdown**, crie uma aplicação de console utilizando **.NET framework**. Se houver dúvidas de como criar projetos utilizando **.NET** no **VSCode** consulte:
-